



FH

University of
Applied Sciences

TECHNIKUM

WIEN

Moderne Roboterkonzepte
Implementierung einer sprachlichen ROS2-Schnittstelle
mittels LLM, RAG und LLM-Agenten-Tools

Einführung

- LLMs immer präsenter, in den meisten Fällen aber in ihrer geschlossenen Umgebung ohne physische Auswirkungen
- Durch Einsatz von LLM-Agents: LLMs können Tools aufrufen, welche die Schnittstelle zur realen Welt darstellen
- Einfluss in der Robotik: Robotersteuerung durch Instruktionen an den Roboter in natürlicher Sprache

Aufgabenstellung



- Implementierung eines Chatbots zur Steuerung einer Roboteranwendung (LLM-Interface, RAG, History, Agent, Tooling)
- LLM muss Userprompt selbstständig interpretieren und die Infos zur Zielpose dann aus der Angabe extrahieren
- System (bzw. Agent) muss selbstständig entscheiden, wann welches Tool aufgerufen wird
- Input: natürliche Sprache („Fahr auf B“)
- Output: Publishen von Koordinaten auf ROS2 Topic (->Roboter fährt auf B)

Setup

- LLMs:

-  Google Gemini-2.5-flash (begrenzt Tokens)

-  Mistral-small-latest



*Schnittstelle User
und System*

- Langchain-Orchestrierungsframework

- Funktionen zum Erstellen von Agents, welche Tools aufrufen können
 - Funktionen zum Erstellen des RAG-Systems & der Tools
 - Funktionen zum formatieren der Inputs (User&Systemprompt, History) & Outputs (Antwort des LLMs) für Agent



*Schnittstelle LLM,
RAG & Tools*

- ROS2 (Humble):

- Simulation des Turtlebot3 in Gazebo
 - Ansteuerung des Turtlebot3 durch das LLM

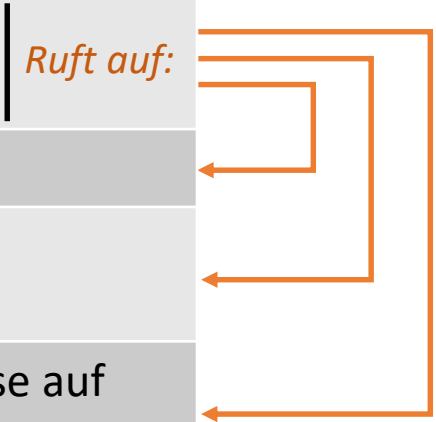


*Schnittstelle Tools &
(simulierter) Roboter*

- Vollständige Implementierung in einem Docker-Container

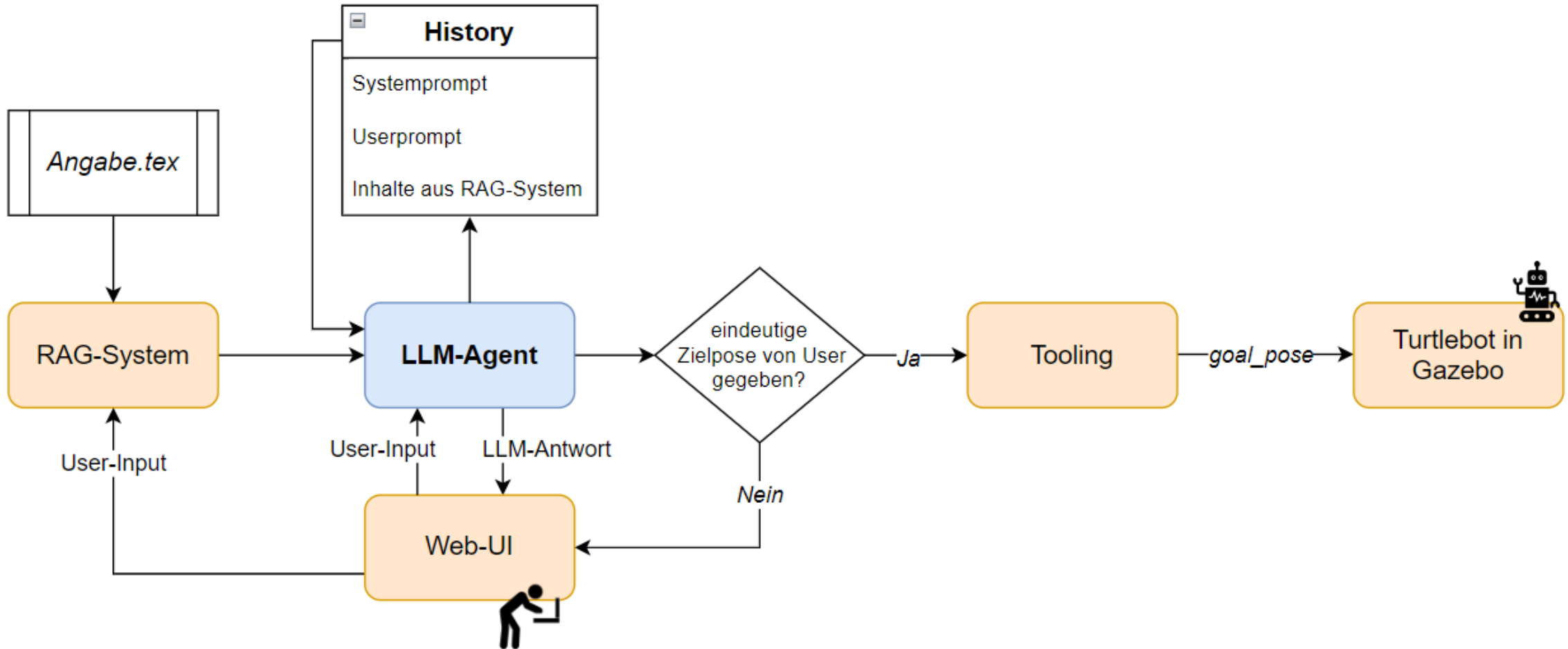
Systemaufbau

Filename	Inhalt der Datei
app.py	Frontend: Interaktion mit User über GUI
Chatbot.py	Main-Backend-File: legt LLM-Agent an, verarbeitet Interaktion mit User
RAG.py	Implementierung des RAG-Systems
ROS_tools.py	Implementierung des LLM-Agent-Tools „ROS_send_goal“
ROS_cmd_vel_publisher.py	Publishen der vom Tool generierten Zielpose auf das Topic „goal_pose“

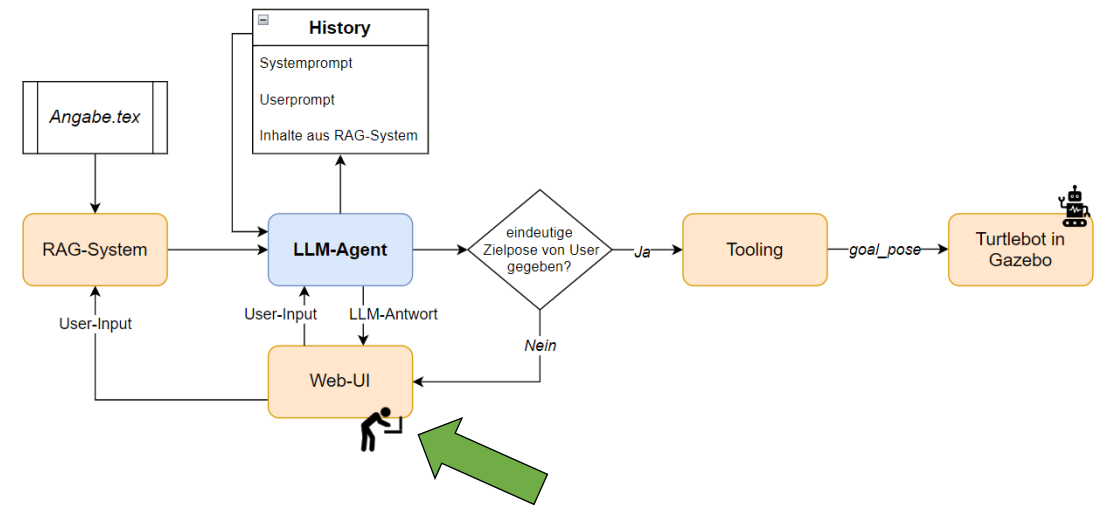


The diagram illustrates the dependencies of the `Chatbot.py` file. An orange line originates from the right side of the `Chatbot.py` row and branches into three arrows pointing to the right sides of the `RAG.py`, `ROS_tools.py`, and `ROS_cmd_vel_publisher.py` rows. The label *Ruft auf:* (Calls upon) is placed next to the initial branch point of the line.

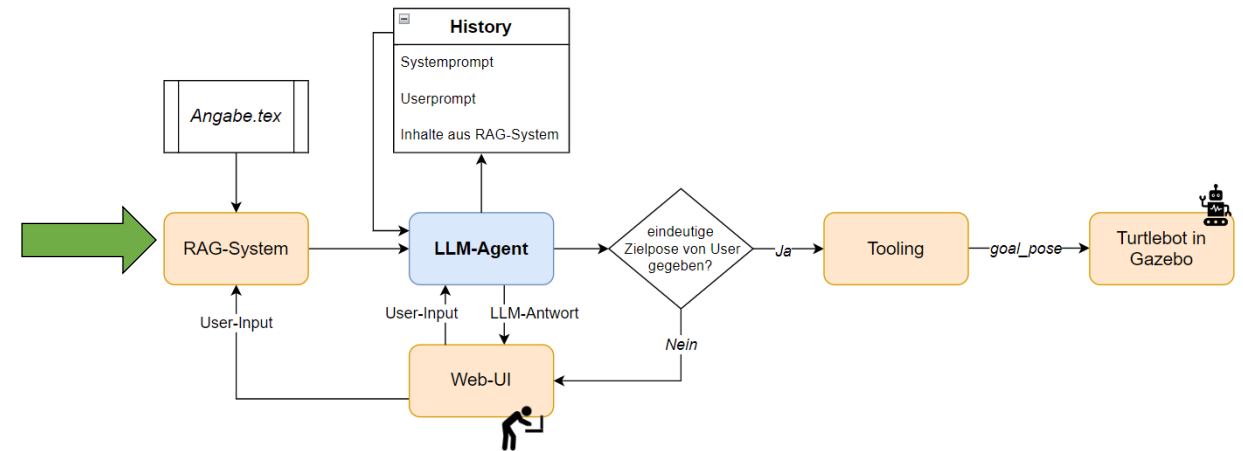
Systemaufbau



User-Interface



RAG-System



1. Chunking

- Splittet den Fließtext der Angabe.tex in Chunks auf

2. Word Embeddings

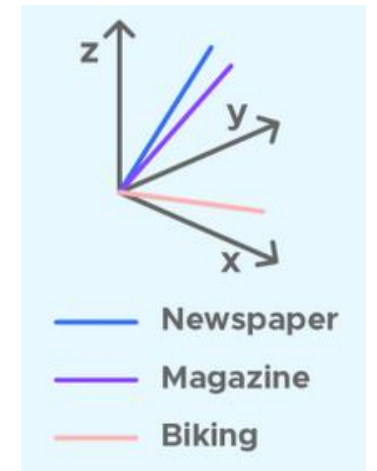
- Darstellung der Chunks im Vektorraum

3. Vector Storage

- Vektordatenbank (FAISS) anlegen, Chunks (in Vektordarstellung) in Vektordatenbank laden

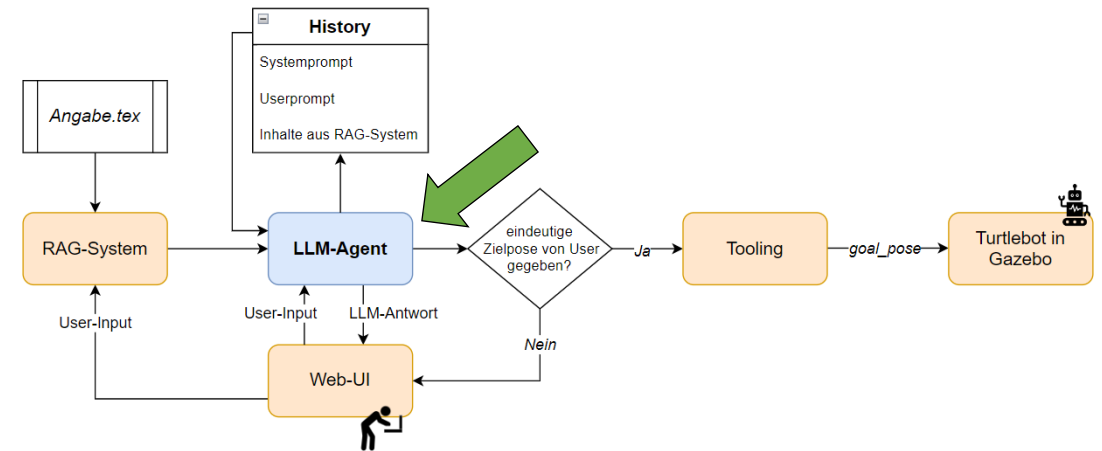
4. Querly

- Userinput in Vektordarstellung (Word Embedding) umwandeln (NICHT zerteilen!)
- Ähnliche Vektoren in Vektordatenbank suchen und zurückgeben

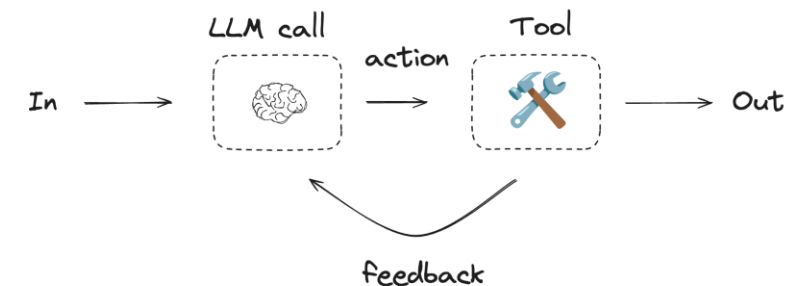


→ Übergabe der Ähnlichen Chunks in History des LLM-Agents ←

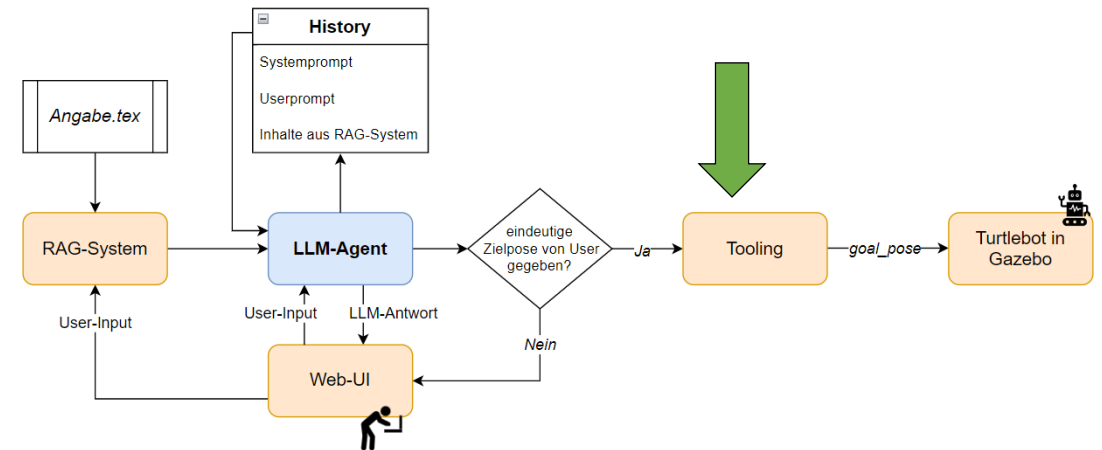
LLM-Agent



1. LLM initialisieren
2. Vollständigen Prompt anlegen (Systemprompt, Platzhalter für Userprompt, Historie)
3. LLM_Agent initialisieren (LLM, Tools, full_prompt)
4. LLM_agent_executor anlegen
5. Ausführen des Agents
 - Führt LLM_Agent aus, verwendet Tools basierend auf Systemprompt, Historie und Userprompt-interpretation durch LLM



Tooling



- **Tools ermöglichen dem LLM-Agent das Handeln über die Sprachfunktion hinaus! (Rechnen, ...)**
- **Toolaufruf durch LLM-Agent:**
 - Aufruf erfolgt erst, wenn LLM eindeutige Pose von User erhält & das RAG Informationen dazu liefert → Wenn Nicht: LLM iteriert und fragt User nochmal nach eindeutiger Pose
 - Toolaufruf durch LLM-Agent basierend auf Tool-Beschreibung (Textform)

Folgende Tools wurden implementiert:

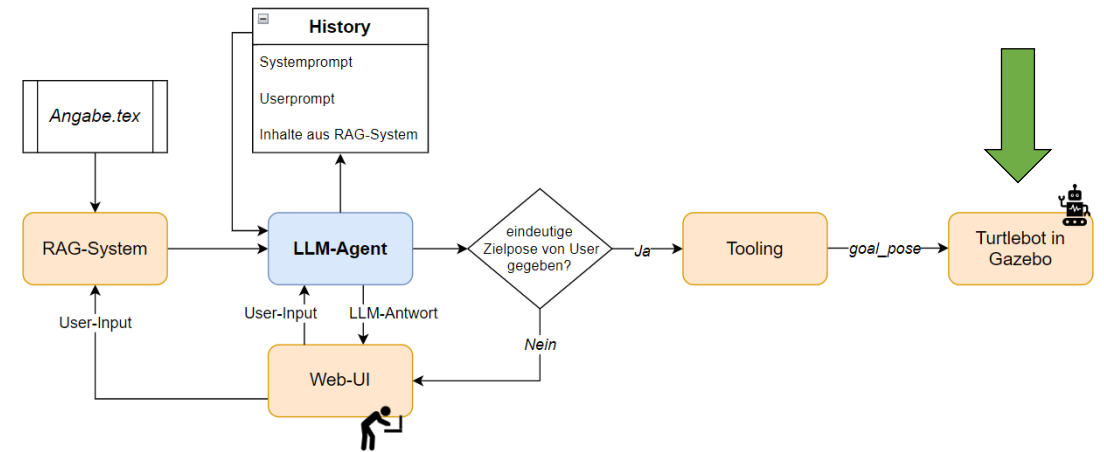
✂ ROS_send_goal

- Übergabe von x , y & θ durch LLM
- Ruft Publisher-Funktion auf und übergibt die x , y und θ

✂ ROS_get_navigation_status

- Überprüft, ob Roboter sein Ziel erreicht hat
- Gibt Status des Roboters zurück

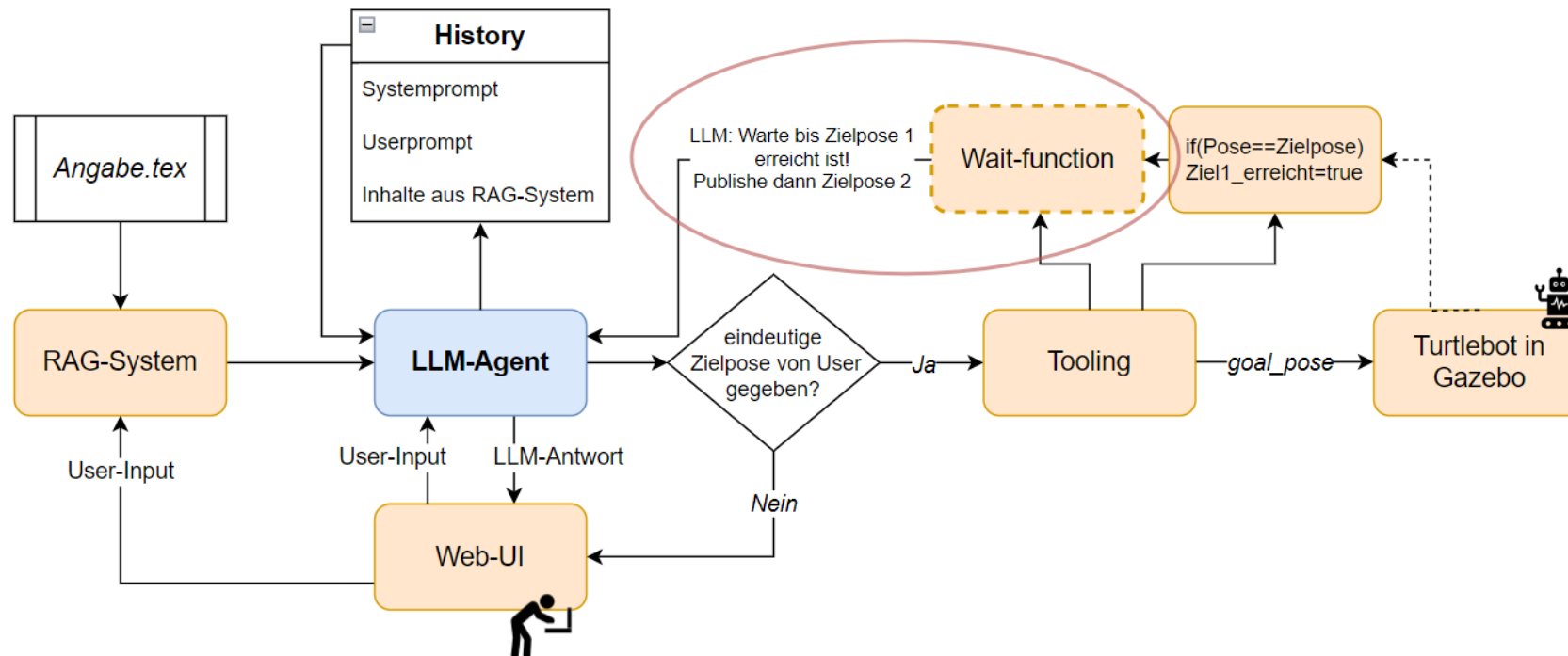
Gazebo-Simulation



Grenzen des Systems

- **Eingabe mehrerer Posen**

- LLM published beide Posen direkt nacheinander und wartet **nicht** bis die Flag von Pose 1 kommt, dass Ziel 1 erreicht wurde
- Führt dazu, dass die Navigation zu Ziel 1 Abgebrochen wird und direkt Ziel 2 angesteuert wird



Demonstration

- Hier Video einfügen, Live-Demo folgt sowieso noch aber falls Demo warum auch immer nicht geht, hier Backup

Zusammenfassung